

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Τεχνολογίες Εφαρμογών Διαδικτύου — ΣΤ' Εξάμηνο, 2026

StayApp

*Εφαρμογή Διαχείρισης Εκδηλώσεων
και Ηλεκτρονικών Κρατήσεων στον Παγκόσμιο Ιστό*

Γεώργιος Πατσάκας — AM sdi2200144

Ανδρέας Λαμπρόπουλος — AM sdi2200252

Διδάσκων: Δρ. Γ. Χαμόδρακας
Σεπτέμβριος 2026

Πίνακας Περιεχομένων

1. Εισαγωγή
Στόχος της εργασίας · Κατανομή αρμοδιοτήτων · Δομή του εγγράφου
2. Αρχιτεκτονική του συστήματος
Επισκόπηση · Τεχνολογίες · Ροή ενός αιτήματος
3. Σχεδίαση της βάσης δεδομένων
Οντότητες και σχέσεις · Από το DTD στο σχεσιακό μοντέλο · Οι δύο μεταναστεύσεις
4. Το REST API ως συμβόλαιο
Γιατί γράφτηκε πρώτο · Εγκάρσιες συμβάσεις · Κατάλογος endpoints
5. Ασφάλεια και εξουσιοδότηση
SSL/TLS · JWT · Ρόλοι ανά ενέργεια · Ταυτόχρονη πρόσβαση
6. Το μετωπιαίο άκρο (React)
Οργάνωση · Ανάπτυξη mock-first · Σύστημα σχεδίασης
7. Ο αλγόριθμος συστάσεων
Biased Matrix Factorization · Implicit feedback · Cold start · Αξιολόγηση στο dataset του e-class
8. Εξαγωγή δεδομένων σε XML και JSON
9. Εγκατάσταση και εκτέλεση
10. Σχεδιαστικές αποφάσεις και παραδοχές
11. Επίλογος — δυσκολίες και πώς αντιμετωπίστηκαν

1. Εισαγωγή

1.1 Στόχος της εργασίας

Η *StayApp* είναι εφαρμογή διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων εισιτηρίων, υλοποιημένη κατά την αρχιτεκτονική web browser / web server. Κάθε εγγεγραμμένος χρήστης λειτουργεί ταυτόχρονα ως **διοργανωτής** στις εκδηλώσεις που δημιουργεί και ως **συμμετέχων** σε όσες κρατά θέση, ενώ ο μη συνδεδεμένος **επισκέπτης** μπορεί μόνο να πλοηγηθεί και να αναζητήσει. Ο **διαχειριστής** εγκρίνει ή απορρίπτει τις αιτήσεις εγγραφής και εξάγει τα δεδομένα των εκδηλώσεων.

Πέρα από τις λειτουργικές απαιτήσεις, η εργασία υλοποιεί σύστημα συστάσεων με αλγόριθμο **Biased Matrix Factorization** γραμμένο εκ του μηδενός, χωρίς χρήση έτοιμων βιβλιοθηκών μηχανικής μάθησης.

1.2 Κατανομή αρμοδιοτήτων

Η ομάδα εργάστηκε σε δύο ανεξάρτητα άκρα που συναντήθηκαν σε ένα ρητό συμβόλαιο διεπαφής. Το μοντέλο αυτό επέτρεψε να προχωρούν και τα δύο μέρη παράλληλα, χωρίς το ένα να περιμένει το άλλο.

Μέλος	Αρμοδιότητα
Γεώργιος Πατσάκας	Μετωπιαίο άκρο (React), σχεδίαση διεπαφής, κατανάλωση του REST API, σύνταξη του αρχικού συμβολαίου <code>API_CONTRACT.md</code> .
Ανδρέας Λαμπρόπουλος	Νωτιαίο άκρο (Python / FastAPI), σχεσιακή βάση δεδομένων και ORM, αλγόριθμος συστάσεων, εξαγωγή XML/JSON.

1.3 Δομή του εγγράφου

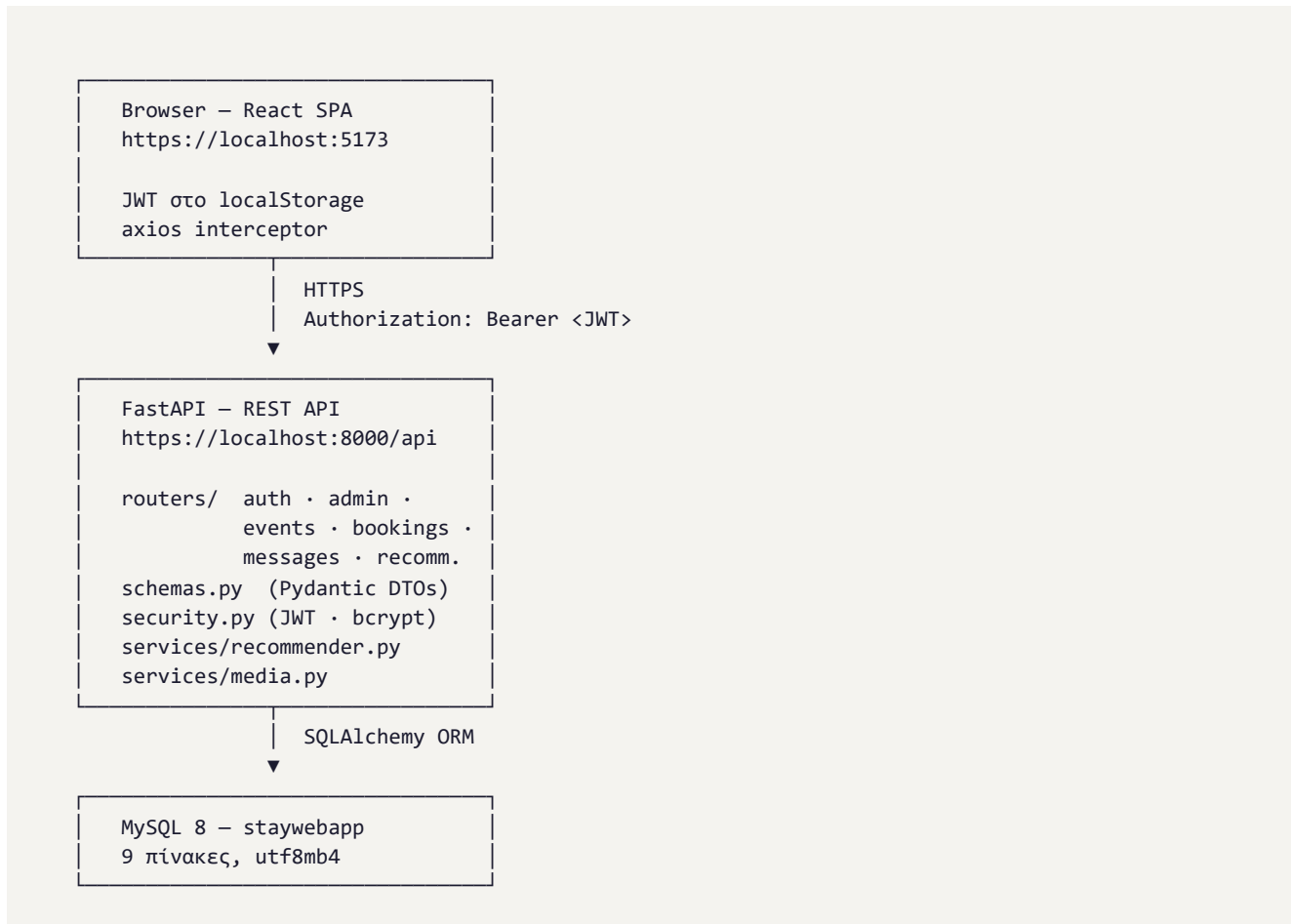
Το **Κεφάλαιο 2** περιγράφει τη συνολική αρχιτεκτονική και τις τεχνολογίες. Το **Κεφάλαιο 3** αναλύει τη σχεδίαση της βάσης δεδομένων και τον τρόπο που το DTD της εκφώνησης μεταφράστηκε σε σχεσιακό μοντέλο. Το **Κεφάλαιο 4** παρουσιάζει το REST API ως συμβόλαιο μεταξύ των δύο άκρων. Το **Κεφάλαιο 5** καλύπτει την ασφάλεια, την εξουσιοδότηση και τον χειρισμό ταυτόχρονων κρατήσεων. Το **Κεφάλαιο 6** αφορά το μετωπιαίο άκρο και το **Κεφάλαιο 7** τον αλγόριθμο συστάσεων. Τα **Κεφάλαια 8 και 9** καλύπτουν την εξαγωγή δεδομένων και τις οδηγίες εγκατάστασης. Το **Κεφάλαιο 10**

συγκεντρώνει τις σχεδιαστικές αποφάσεις και τις παραδοχές, και ο **Επίλογος** τις δυσκολίες που συναντήσαμε.

2. Αρχιτεκτονική του συστήματος

2.1 Επισκόπηση

Η εφαρμογή χωρίζεται σε τρία επίπεδα που επικοινωνούν αποκλειστικά μέσω κρυπτογραφημένων συνδέσεων: μια μονοσέλιδη εφαρμογή (SPA) στον browser, ένα REST API, και μια σχεσιακή βάση δεδομένων.



Σχήμα 2.1 — Τα τρία επίπεδα της εφαρμογής.

2.2 Τεχνολογίες και αιτιολόγηση

Επίπεδο	Τεχνολογία	Γιατί
Μετωπιαίο άκρο	React 18, Vite 6, React Router 6	Η εκφώνηση (§6 Λοιπών) απαιτεί JavaScript web framework. Το Vite δίνει άμεσο dev server και ενσωματωμένο proxy προς το API.
Νωτιαίο άκρο	FastAPI, Pydantic v2, Uvicorn	Τα Pydantic σχήματα επικυρώνουν την είσοδο και παράγουν τα DTO εξόδου από ένα σημείο, ενώ το <code>/docs</code> δίνει δωρεάν διαδραστική τεκμηρίωση.
ORM	SQLAlchemy 2	Η εκφώνηση απαιτεί απεικόνιση του σχεσιακού μοντέλου σε αντικειμενοστρεφές.
Βάση	MySQL 8 (utf8mb4)	Υποστήριξη <code>SELECT ... FOR UPDATE</code> , απαραίτητη για την αποφυγή υπερκρατήσεων (§5.4).
Ασφάλεια	TLS, PyJWT, bcrypt	Η εκφώνηση §1 απαιτεί κρυπτογράφηση όλων των αλληλεπιδράσεων· οι κωδικοί δεν αποθηκεύονται ποτέ σε καθαρό κείμενο.

2.3 Η ροή ενός αιτήματος

Παράδειγμα: ο χρήστης υποβάλλει κράτηση. Το component καλεί τη συνάρτηση του API facade, ποτέ απευθείας τον axios. Ο interceptor προσθέτει το JWT. Το αίτημα φτάνει στον router, όπου το Pydantic έχει ήδη επικυρώσει το σώμα. Ο router κλειδώνει τη γραμμή της εκδήλωσης και του τύπου εισιτηρίου, ελέγχει διαθεσιμότητα, εγγράφει την κράτηση και μειώνει το `available` μέσα στην ίδια συναλλαγή. Το αποτέλεσμα επιστρέφει ως DTO.

Η ουσιώδης αρχή είναι ότι **η διαθεσιμότητα αποφασίζεται πάντα από τον server**. Το μετωπιαίο άκρο εμφανίζει αριθμούς, δεν τους υπολογίζει: δύο χρήστες που κρατούν την ίδια στιγμή θα πάρουν σωστό αποτέλεσμα επειδή ο έλεγχος γίνεται κάτω από κλείδωμα στη βάση, όχι στον browser.

3. Σχεδίαση της βάσης δεδομένων

3.1 Οντότητες και σχέσεις

Η βάση αποτελείται από εννέα πίνακες. Ο πυρήνας είναι η εκδήλωση: γύρω της οργανώνονται οι τύποι εισιτηρίων, οι κρατήσεις, οι κατηγορίες, το οπτικό υλικό, τα μηνύματα και οι επισκέψεις.

Πίνακας	Σχέση	Περιγραφή
users	—	Χρήστες με ΑΦΜ, τοποθεσία, <code>status</code> και <code>role</code> .
events	N:1 → users	Εκδηλώσεις· <code>organizer_id</code> ο δημιουργός.
ticket_types	N:1 → events	Τύποι εισιτηρίων με τιμή, ποσότητα και διαθέσιμο υπόλοιπο.
bookings	N:1 → users, events, ticket_types	Κρατήσεις με πλήθος, συνολικό κόστος και κατάσταση.
event_categories	—	Κατάλογος μοναδικών ονομάτων κατηγοριών.
event_has_categories	N:M γέφυρα	Συνδέει εκδηλώσεις με κατηγορίες.
event_media	N:1 → events	Ονόματα αρχείων οπτικού υλικού.
messages	N:1 → users (×2), events	Μηνύματα με αποστολέα, παραλήπτη και υποχρεωτική εκδήλωση.
event_visits	N:1 → users, events	Επισκέψεις σε σελίδα εκδήλωσης — τα δεδομένα cold start του §13.

3.2 Από το DTD στο σχεσιακό μοντέλο

Το DTD της εκφώνησης (§7α) περιγράφει ιεραρχικό έγγραφο. Η μετάφρασή του σε σχεσιακό σχήμα απαίτησε τρεις αποφάσεις:

- Τα επαναλαμβανόμενα στοιχεία (`Category+` , `TicketType+` , `Booking*` , `Photo*`) έγιναν ξεχωριστοί πίνακες με ξένο κλειδί προς την εκδήλωση.
- Το προαιρετικό `GeoLocation` έγινε δύο `NULL`-able στήλες. Όταν λείπουν συντεταγμένες το API επιστρέφει `geoLocation: null` και όχι `{lat:0, lng:0}` — το σημείο (0,0) είναι

υπαρκτή τοποθεσία στον Ατλαντικό και θα εμφάνιζε ψευδή πινέζα στον χάρτη.

- Το `EventID="EV1024"` του DTD αφορά **μόνο την εξαγωγή XML**. Εσωτερικά τα αναγνωριστικά είναι ακέραιοι αυτόματης αύξησης.

3.3 Μετανάστευση 001 — κατηγορίες από 1:N σε N:M

Στο αρχικό σχήμα ο πίνακας `event_categories` είχε στήλη `event_id NOT NULL`: κάθε γραμμή κατηγορίας ανήκε σε μία εκδήλωση. Αυτό σήμαινε ότι πενήντα συναυλίες θα παρήγαγαν πενήντα ξεχωριστές γραμμές με όνομα «Music», κάνοντας το φίλτρο `?category=Music` της §8 αδύνατο να λειτουργήσει σωστά.

Η μετανάστευση αποσυνδέει τον κατάλογο κατηγοριών από τη συγκεκριμένη εκδήλωση, επιβάλλει μοναδικότητα στο όνομα και εισάγει τον πίνακα-γέφυρα `event_has_categories`. Το πρωτόκολλο προς το μετωπιαίο άκρο **δεν άλλαξε**: το πεδίο `categories` παραμένει απλός πίνακας συμβολοσειρών και ο server κάνει *get-or-create* κατά τη δημιουργία.

3.4 Μετανάστευση 002 — διαγραφή μηνυμάτων ανά χρήστη

Η εκφώνηση §10 απαιτεί οι χρήστες να διαγράφουν μηνύματα από τους καταλόγους εισερχομένων και απεσταλμένων. Η ίδια γραμμή του `messages` όμως εξυπηρετεί **και τις δύο πλευρές**: είναι το εισερχόμενο του παραλήπτη και ταυτόχρονα το απεσταλμένο του αποστολέα.

ΤΟ ΠΡΟΒΛΗΜΑ

Μια απλή εντολή `DELETE` θα έσβηνε το μήνυμα και από τον συνομιλητή. Ο ένας χρήστης θα μπορούσε δηλαδή να εξαφανίσει περιεχόμενο από τον κατάλογο του άλλου — συμπεριφορά που καμία εφαρμογή μηνυμάτων δεν επιτρέπει.

Η λύση είναι δύο ανεξάρτητες σημαίες, `deleted_by_sender` και `deleted_by_receiver`. Ο καθένας κρύβει το μήνυμα μόνο από τη δική του όψη. Όταν το κρύψουν και οι δύο, η γραμμή δεν είναι πλέον ορατή σε κανέναν και το API τη διαγράφει οριστικά, ώστε ο πίνακας να μη γεμίζει με άορατα δεδομένα. Δύο ευρετήρια, στα `(to_user_id, deleted_by_receiver)` και `(from_user_id, deleted_by_sender)`, εξυπηρετούν τους δύο καταλόγους και το `unread-count` που το μετωπιαίο άκρο ζητά κάθε 30 δευτερόλεπτα.

4. Το REST API ως συμβόλαιο

4.1 Γιατί γράφτηκε πρώτο

Πριν γραφτεί γραμμή κώδικα σε οποιοδήποτε άκρο, συντάχθηκε το αρχείο `API_CONTRACT.md` : μια πλήρης προδιαγραφή των endpoints, των DTO και των κωδικών σφάλματος. Ο λόγος ήταν πρακτικός. Με δύο άτομα να δουλεύουν παράλληλα, κάθε σιωπηλή απόκλιση του ενός εμφανιζόταν στον άλλον ως ανεξήγητο σφάλμα. Το συμβόλαιο έγινε η μοναδική πηγή αλήθειας, με ρητό κανόνα ότι καμία αλλαγή δεν γίνεται χωρίς συμφωνία και καταγραφή στο changelog.

Η πρακτική αυτή απέδωσε: το μετωπιαίο άκρο αναπτύχθηκε εξ ολοκλήρου πάνω σε εικονικά δεδομένα (§6.2) και, όταν ήρθε η ώρα της ένωσης, οι δύο πλευρές ταίριαξαν σε επίπεδο πεδίων χωρίς αλλαγές στα DTO.

4.2 Εγκάρσιες συμβάσεις

Οι παρακάτω κανόνες ισχύουν σε όλα τα endpoints.

Τυποποιημένη μορφή σφάλματος

Κάθε σφάλμα, χωρίς εξαίρεση, επιστρέφεται στην ίδια μορφή. Δύο exception handlers στο `main.py` κανονικοποιούν ακόμα και τα σφάλματα του ίδιου του framework, ώστε ένα 404 σε άγνωστη διαδρομή να μη διαφέρει από ένα 404 του δικού μας κώδικα:

```
{ "error": { "code": "SEATS_UNAVAILABLE", "message": "Εξαντλήθηκαν οι θέσεις." } }
```

Επικύρωση: 400, ποτέ 422

Το Pydantic παράγει από προεπιλογή 422. Ο αντίστοιχος handler το μετατρέπει σε **400** `VALIDATION_ERROR` με χάρτη πεδίου προς μήνυμα. Οι διαδρομές είναι *dotted* και περιλαμβάνουν δείκτες λίστας, ώστε το μετωπιαίο άκρο να τοποθετεί το σφάλμα κάτω από το σωστό πεδίο της φόρμας:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "...",  
  "details": {  
    "capacity": "Input should be greater than or equal to 1",  
    "ticketTypes.0.quantity": "Input should be greater than or equal to 1"  
  } } }
```

Σφάλματα που αφορούν σχέση δύο πεδίων — για παράδειγμα «η λήξη πρέπει να είναι μετά την έναρξη» — δεν χρεώνονται σε συγκεκριμένο πεδίο και έρχονται με κλειδί `"body"`, ώστε να εμφανίζονται ως μήνυμα πάνω από τη φόρμα.

Σελιδοποίηση

Κάθε λίστα δέχεται `?page=1&pageSize=20` και επιστρέφει τον ίδιο περιτυλιγμένο τύπο με `items`, `page`, `pageSize`, `total` και `totalPages`. Σελίδα πέρα από το τέλος επιστρέφει **200 με κενή λίστα**, όχι 404: η αναζήτηση δεν απέτυχε, απλώς δεν έχει άλλα αποτελέσματα.

Χρήματα και ημερομηνίες

Τα ποσά ταξιδεύουν ως **συμβολοσειρά με δύο δεκαδικά** (`"18.00"`), ώστε να μην εισαχθούν σφάλματα κινητής υποδιαστολής στη σειριοποίηση. Οι ημερομηνίες είναι **ISO-8601 σε UTC με κατάληξη `Z`**. Μοναδική εξαίρεση η εξαγωγή XML, που γράφει ημερομηνίες χωρίς `Z` επειδή έτσι τις ορίζει το DTD της εκφώνησης.

4.3 Κατάλογος endpoints

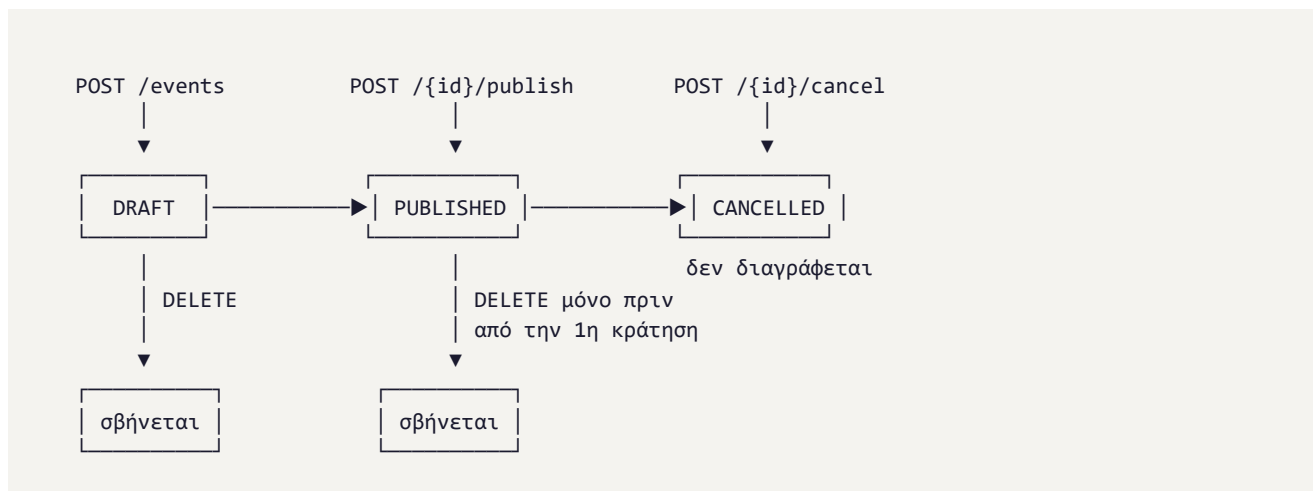
Υλοποιήθηκαν 28 endpoints, οργανωμένα σε έξι ενότητες.

Ενότητα	Πλήθος	Ενδεικτικά
Ταυτοποίηση	2	<code>POST /auth/register</code> , <code>POST /auth/login</code>
Διαχείριση	6	<code>GET /admin/users</code> , <code>POST /admin/users/{id}/approve</code> , <code>GET /admin/events/export</code>
Εκδηλώσεις	10	<code>GET /events</code> , <code>POST /events</code> , <code>POST /events/{id}/publish</code> , <code>POST /events/{id}/cancel</code> , <code>POST /events/{id}/media</code>
Κρατήσεις	2	<code>POST /bookings</code> , <code>GET /bookings/mine</code>
Μηνύματα	7	<code>GET /messages/inbox</code> , <code>GET /messages/unread-count</code> , <code>DELETE /messages/{id}</code>
Συστάσεις	1	<code>GET /recommendations</code>

4.4 Ο κύκλος ζωής μιας εκδήλωσης

Μια εκδήλωση γεννιέται σε κατάσταση `DRAFT`: δεν εμφανίζεται στην αναζήτηση και δεν δέχεται κρατήσεις. Απαιτείται ρητή δημοσίευση. Η κατάσταση αλλάζει **μόνο** μέσω των

`/publish` και `/cancel` — η ενημέρωση με `PUT` δεν την αγγίζει ποτέ, ώστε μια αθώα επεξεργασία να μην μπορεί να δημοσιεύσει κατά λάθος.



Σχήμα 4.1 — Καταστάσεις εκδήλωσης και οι επιτρεπτές μεταβάσεις.

Η διαγραφή ακολουθεί κατά γράμμα την §7γ: επιτρέπεται «μόνο πριν από τη δημοσίευση της εκδήλωσης ή, το αργότερο, πριν από την υποβολή της πρώτης κράτησης». Άρα διαγράφεται και μια δημοσιευμένη εκδήλωση, αρκεί να μην έχει ακόμη κράτηση. Η ακυρωμένη όχι, αφού η ακύρωση ορίζεται ρητά ως μεταβολή κατάστασης «χωρίς διαγραφή των αποθηκευμένων δεδομένων». Ο κανόνας είναι γραμμένος σε ένα σημείο (ιδιότητα `is_deletable` του μοντέλου), από το οποίο τροφοδοτούνται τόσο το πεδίο `isDeletable` του DTO όσο και ο έλεγχος του `DELETE`, ώστε η διεπαφή και ο server να μην μπορούν να διαφωνήσουν.

Μια δημοσιευμένη εκδήλωση δέχεται κρατήσεις **μόνο έως την ώρα έναρξής της**: η §9 επιτρέπει κρατήσεις «όταν η εκδήλωση είναι ενεργή», και μια εκδήλωση που έχει ήδη ξεκινήσει δεν είναι, ακόμη κι αν κανείς δεν έχει αλλάξει την κατάστασή της.

Η ακύρωση εκτελείται σε **μία συναλλαγή**: η κατάσταση γίνεται `CANCELLED` και ταυτόχρονα αποστέλλεται ενημερωτικό μήνυμα σε κάθε μοναδικό χρήστη με ενεργή κράτηση, όπως απαιτεί η §10. Ένα μήνυμα ανά χρήστη και όχι ανά κράτηση, με αποστολέα τον διοργανωτή ώστε η απάντηση να πάει στον σωστό παραλήπτη. Οι κρατήσεις **διατηρούνται** για λόγους ιστορικότητας και ιχνηλασιμότητας, όπως ορίζει η εκφώνηση.

5. Ασφάλεια και εξουσιοδότηση

5.1 Κρυπτογράφηση της μεταφοράς

Η εκφώνηση §1 απαιτεί όλες οι αλληλεπιδράσεις να είναι κρυπτογραφημένες. Και τα δύο άκρα εξυπηρετούνται μέσω TLS: το API στο `https://localhost:8000` και ο dev server του μετωπιαίου άκρου στο `https://localhost:5173`, χρησιμοποιώντας κοινό αυτο-υπογεγραμμένο πιστοποιητικό που παράγεται από το `make-certs.sh`. Οι κλήσεις προς `/api` δρομολογούνται μέσω του proxy του Vite, οπότε ο browser τις βλέπει ως ίδιας προέλευσης.

5.2 Κωδικοί και JWT

Οι κωδικοί αποθηκεύονται κατακερματισμένοι με **bcrypt** και δεν επιστρέφονται ποτέ σε κανένα DTO. Μετά την επιτυχή είσοδο εκδίδεται JSON Web Token με ωφέλιμο φορτίο:

```
{ "sub": "12", "username": "maria21", "role": "USER", "exp": ... }
```

Το πεδίο `sub` είναι **συμβολοσειρά**, όπως επιβάλλει το πρότυπο JWT, παρότι εσωτερικά τα αναγνωριστικά είναι ακέραιοι. Η λεπτομέρεια καταγράφηκε ρητά στο σύμβολο, γιατί μια σύγκριση `===` με ακέραιο στο μετωπιαίο άκρο θα αποτύγχανε σιωπηλά σε κάθε έλεγχο ιδιοκτησίας.

5.3 Ρόλοι ανά ενέργεια, όχι ανά χρήστη

Η εκφώνηση ορίζει τέσσερις ρόλους, αλλά ο «διοργανωτής» και ο «συμμετέχων» **δεν είναι σταθερές ιδιότητες**: ο ίδιος χρήστης είναι διοργανωτής στις εκδηλώσεις που δημιουργεί και συμμετέχων σε όσες κρατά. Το JWT κρατά μόνο `ADMIN` ή `USER`· τα υπόλοιπα δικαιώματα κρίνονται ανά πόρο, από την ιδιοκτησία.

Κωδικός	Σημασία	Αντίδραση της διεπαφής
401	Λείπει, έληξε ή είναι άκυρο το JWT	Καθαρισμός συνεδρίας και ανακατεύθυνση στην είσοδο
403	Έγκυρο token, λάθος ρόλος ή μη ιδιοκτήτης	Μήνυμα σφάλματος — χωρίς αποσύνδεση

Η διάκριση δεν είναι ακαδημαϊκή. Αν η διεπαφή αποσύνδεε τον χρήστη και στις δύο περιπτώσεις, θα τον πετούσε έξω κάθε φορά που πατούσε κάτι που δεν του ανήκει.

Ξεχωριστή περίπτωση αποτελεί η πρόσβαση σε αδημοσίευτη εκδήλωση άλλου χρήστη: το API επιστρέφει **404** και όχι **403**. Η επιλογή είναι σκόπιμη, αφού το 403 θα επιβεβαίωνε την ύπαρξη ενός πόρου που ο χρήστης δεν δικαιούται να γνωρίζει.

5.4 Ταυτόχρονες κρατήσεις

Ο κίνδυνος της υπερκράτησης εμφανίζεται όταν δύο αιτήματα ελέγξουν τη διαθεσιμότητα πριν προλάβει οποιοδήποτε να γράψει. Η υλοποίηση κλειδώνει τις εμπλεκόμενες γραμμές με `SELECT ... FOR UPDATE` και εκτελεί έλεγχο και εγγραφή μέσα στην ίδια συναλλαγή.

Επιπλέον, τα κλειδώματα λαμβάνονται πάντα με **σταθερή σειρά** — πρώτα η εκδήλωση, έπειτα οι τύποι εισιτηρίων κατά αύξον αναγνωριστικό — τόσο στην κράτηση όσο και στην ενημέρωση εκδήλωσης. Χωρίς αυτή τη σύμβαση, δύο ταυτόχρονες ενέργειες που κλειδώνουν τους ίδιους πόρους με αντίστροφη σειρά θα οδηγούσαν σε αδιέξοδο (deadlock).

5.5 Περιορισμός της επικοινωνίας

Η αποστολή μηνύματος επιτρέπεται μόνο όταν οι δύο πλευρές συνδέονται μέσω συγκεκριμένης εκδήλωσης: ο ένας είναι ο διοργανωτής και ο άλλος έχει ενεργή κράτηση σε αυτήν. Χωρίς τον έλεγχο, οποιοσδήποτε εγκεκριμένος χρήστης θα μπορούσε να στείλει μήνυμα σε οποιονδήποτε άλλον — δηλαδή ανοιχτό κανάλι ανεπιθύμητης αλληλογραφίας.

6. Το μετωπιαίο άκρο

6.1 Οργάνωση

Ο κώδικας οργανώνεται κατά αρμοδιότητα και όχι κατά τύπο αρχείου. Δύο κανόνες τηρούνται απαρέγκλιτα: **καμία κλήση δικτύου δεν γίνεται από component** — όλες περνούν από το `api/index.js` — και **κανένα χρώμα ή διάστημα δεν γράφεται απευθείας μέσα σε CSS component**.

```
src/
  api/          facade προς το API (μοναδικό σημείο κλήσεων)
    client.js    axios instance · JWT interceptor
    mock/        εικονικά δεδομένα ανάπτυξης
  auth/         αποθήκευση JWT (μοναδικό σημείο)
  context/       AuthContext — χρήστης, ρόλος, login/logout
  routes/        ProtectedRoute — περιορισμός ανά ρόλο
  components/    επαναχρησιμοποιήσιμα (events/, ui/, form/, layout/)
  pages/         μία σελίδα ανά διαδρομή
  styles/        theme.css (tokens) · components.css · global.css
```

6.2 Ανάπτυξη mock-first

Επειδή τα δύο άκρα αναπτύσσονταν παράλληλα, το μετωπιαίο άκρο δεν μπορούσε να περιμένει το API. Κάθε συνάρτηση του facade γράφτηκε με **δύο διαδρομές**: μία που απαντά από εικονικά δεδομένα στη μνήμη και μία που καλεί τον πραγματικό server.

```
export async function createBooking({ eventId, ticketTypeId, numberOfTickets }) {
  if (USE MOCK) {
    /* ... προσομοίωση των ελέγχων του server ... */
  }
  const { data } = await client.post('/bookings', { ... })
  return data
}
```

Κρίσιμο σημείο: τα εικονικά δεδομένα προσομοιώνουν και τις **αποτυχίες** με τους ίδιους κωδικούς σφάλματος, όχι μόνο τις επιτυχίες. Έτσι οι οθόνες σφαλμάτων δοκιμάστηκαν πριν καν υπάρξει server. Μετά την ένωση, η επιλογή οδηγείται από μεταβλητή περιβάλλοντος και η προεπιλογή είναι ο πραγματικός server· τα εικονικά δεδομένα αφαιρούνται εντελώς από την τελική δέσμη κατά το χτίσιμο.

6.3 Σύστημα σχεδίασης

Αντί για έτοιμη βιβλιοθήκη components, χρησιμοποιήθηκε καθαρό CSS με μεταβλητές που ορίζουν χρώματα, αποστάσεις, τυπογραφία και ακτίνες γωνιών. Η επιλογή τεκμηριώνεται από την εκφώνηση (§5 Λοιπών), όπου η ποιότητα της οργάνωσης βαθμολογείται: ένα μικρό σύνολο μεταβλητών κρατά τη διεπαφή συνεπή και επιτρέπει αλλαγή θέματος από ένα σημείο.

6.4 Μη αναστρέψιμες ενέργειες

Η κράτηση δεν αναιρείται — δεν υπάρχει endpoint διαγραφής της, κατά την §9. Το παράθυρο επιβεβαίωσης πριν την υποβολή είναι επομένως **λειτουργική απαίτηση** και όχι ευγένεια προς τον χρήστη. Το ίδιο μοτίβο εφαρμόζεται στη δημοσίευση, την ακύρωση και τη διαγραφή μηνύματος.

Παράλληλα, η διεπαφή αποκρύπτει ενέργειες που ο server θα απέρριπτε: το κουμπί διαγραφής εκδήλωσης εμφανίζεται μόνο όταν το πεδίο `isDeletable` είναι αληθές, και το όνομα τύπου εισιτηρίου κλειδώνεται όταν υπάρχουν κρατήσεις (§10.3). Ο έλεγχος γίνεται και στις δύο πλευρές· η διεπαφή δεν αποφασίζει, απλώς αποτρέπει προβλέψιμα σφάλματα.

7. Ο αλγόριθμος συστάσεων

7.1 Το μοντέλο

Η εκφώνηση §13 ζητά **Biased Matrix Factorization** υλοποιημένο εκ του μηδενός. Η πρόβλεψη για τον χρήστη u και την εκδήλωση i δίνεται από:

$$\hat{r}(u,i) = \mu + b_u + b_i + q_i^T \cdot p_u$$

Όρος	Ερμηνεία
μ	Ο γενικός μέσος όρος όλων των βαθμολογιών.
b_u	Προκατάληψη χρήστη — πόσο πιο δραστήριος είναι από τον μέσο όρο.
b_i	Προκατάληψη εκδήλωσης — πόσο πιο δημοφιλής είναι από τον μέσο όρο.
p_u, q_i	Διανύσματα λανθανόντων χαρακτηριστικών, k διαστάσεων.

Οι όροι b_u και b_i είναι ακριβώς αυτό που καθιστά το μοντέλο *biased*. Χωρίς αυτούς, το εσωτερικό γινόμενο θα έπρεπε να εξηγήσει μόνο του και τη δημοτικότητα κάθε εκδήλωσης και τη δραστηριότητα κάθε χρήστη — μεγέθη που δεν έχουν σχέση με το *ταίριασμα* των δύο.

7.2 Εκπαίδευση

Η εκπαίδευση γίνεται με στοχαστική κατάβαση κλίσης. Για κάθε παρατήρηση υπολογίζεται το σφάλμα $e = r - \hat{r}$ και οι παράμετροι κινούνται αντίθετα από την κλίση της συνάρτησης κόστους:

$$\sum (r - \hat{r})^2 + \lambda(b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2)$$

Ο όρος κανονικοποίησης λ τιμωρεί τις μεγάλες τιμές και εμποδίζει το μοντέλο να «αποστηθίσει» τα λίγα διαθέσιμα δεδομένα. Δύο λεπτομέρειες της υλοποίησης αξίζει να σημειωθούν:

- Τα διανύσματα P και Q αρχικοποιούνται με μικρό τυχαίο θόρυβο και **όχι με μηδενικά**. Αν ήταν μηδενικά, το γινόμενο και οι κλίσεις τους θα παρέμεναν μηδέν για πάντα και το μοντέλο δεν θα μάθαινε ποτέ αλληλεπιδράσεις.
- Κατά την ενημέρωση, το p_u αντιγράφεται **πριν** μεταβληθεί. Οι δύο ενημερώσεις είναι ταυτόχρονες στα μαθηματικά· η χρήση του ήδη ενημερωμένου p_u για τον υπολογισμό

του q_i θα έδινε λανθασμένη κλίση.

Υπερπαράμετρος	Τιμή	Σκεπτικό
Λανθάνοντα χαρακτηριστικά (k)	8	Μικρό, γιατί τα δεδομένα είναι ελάχιστα. Στην αξιολόγηση (§7.5) το $k = 16$ δεν έδωσε μετρήσιμο όφελος.
Εποχές	20	Αρχικά 60. Η αξιολόγηση στο dataset του e-class έδειξε ότι με 60 εποχές το μοντέλο αποστηθίζει τα δεδομένα εκπαίδευσης (§7.5).
Ρυθμός μάθησης (η)	0.01	Συντηρητικός, για σταθερή σύγκλιση.
Κανονικοποίηση (λ)	0.1	Αρχικά 0.05. Στην αξιολόγηση η επίδρασή του ήταν μικρή· προτιμήθηκε η ισχυρότερη τιμή.
Αρνητικά δείγματα ανά θετικό	1	Απαραίτητα για την κατάταξη εκδηλώσεων που ο χρήστης δεν έχει δει (§7.3).
Σπόρος τυχειότητας	42	Αναπαραγωγίμα αποτελέσματα: ίδια δεδομένα, ίδιες προτάσεις.

7.3 Implicit feedback

Η εφαρμογή δεν ζητά από τους χρήστες ρητές βαθμολογίες. Η προτίμηση συνάγεται από τη συμπεριφορά: η **επίσκεψη** σε σελίδα εκδήλωσης δείχνει ενδιαφέρον, ενώ η **κράτηση** δείχνει δέσμευση — και κοστίζει χρήματα, γι' αυτό βαραίνει πενταπλάσια.

Σήμα	Βάρος	Πηγή
Επίσκεψη σε εκδήλωση	1.0	<code>event_visits</code>
Κράτηση	5.0	<code>bookings</code> (πλην ακυρωμένων)
Ανώτατο όριο βαθμολογίας	10.0	—

Το ανώτατο όριο δεν είναι διακοσμητικό: χωρίς αυτό, ένας χρήστης που άνοιξε σαράντα φορές την ίδια σελίδα θα κυριαρχούσε στην εκπαίδευση και θα στρέβλωνε όλες τις προκαταλήψεις. Όταν ο ίδιος χρήστης και επισκέφθηκε και κράτησε την ίδια εκδήλωση, τα δύο σήματα αθροίζονται σε μία βαθμολογία.

Αρνητικά δείγματα

Όλες οι παρατηρήσεις της εφαρμογής είναι **θετικές**: δεν υπάρχει «δεν μου αρέσει». Αυτό κρύβει μια παγίδα. Ένα μοντέλο παλινδρόμησης που βλέπει παντού «του άρεσε» μαθαίνει

να προβλέπει «του αρέσει» για τα πάντα, οπότε δεν μπορεί να ξεχωρίσει ποιες άγνωστες εκδηλώσεις ταιριάζουν στον χρήστη — και αυτές ακριβώς είναι οι υποψήφιες προτάσεις.

Το πρόβλημα εντοπίστηκε με ελεγχόμενο πείραμα: έξι χρήστες κράτησαν τέσσερις μουσικές εκδηλώσεις, έξι άλλοι τέσσερις αθλητικές, και μια χρήστρια δύο από τις μουσικές. Όλες οι εκδηλώσεις είναι εξίσου δημοφιλείς, άρα μόνο τα λανθάνοντα χαρακτηριστικά μπορούν να δείξουν ότι της ταιριάζουν οι υπόλοιπες μουσικές. Χωρίς αρνητικά δείγματα, το μοντέλο τις έβαζε στις δύο πρώτες θέσεις το πολύ σε 2 από 10 εκτελέσεις — ουσιαστικά τυχαία.

Η λύση είναι η τυπική για παραγοντοποίηση σε implicit feedback: σε κάθε εποχή, για τυχαίες θετικές παρατηρήσεις κρατάμε τον χρήστη και διαλέγουμε τυχαία μια εκδήλωση που **δεν έχει δει**, με στόχο βαθμολογία 0 — μια αθέατη εκδήλωση είναι ασθενής ένδειξη αδιαφορίας. Τα δείγματα αλλάζουν σε κάθε εποχή, ώστε καμία συγκεκριμένη εκδήλωση να μην «τιμωρείται» μόνιμα. Με ένα αρνητικό ανά θετικό και 20 εποχές, το ίδιο πείραμα έδωσε σωστή κατάταξη σε **10 από 10** εκτελέσεις.

ΠΩΣ ΣΥΛΛΕΓΟΝΤΑΙ ΟΙ ΕΠΙΣΚΕΨΕΙΣ

Η καταγραφή γίνεται σιωπηρά στο `GET /events/{id}`, για συνδεδεμένο χρήστη που δεν είναι ο διοργανωτής, σε δημοσιευμένη εκδήλωση. Οι επισκέψεις του ίδιου του διοργανωτή εξαιρούνται ρητά, γιατί θα μόλυναν τις προτάσεις του με τις δικές του εκδηλώσεις. Στο μετωπιαίο άκρο, η κλήση γίνεται **μία φορά ανά πραγματικό άνοιγμα σελίδας**: το παράθυρο σύνταξης μηνύματος, για παράδειγμα, αποφεύγει σκόπιμα να ξαναζητήσει την εκδήλωση.

7.4 Ψυχρή εκκίνηση

Η εκφώνηση ορίζει ότι, αν ο χρήστης δεν έχει ιστορικό κρατήσεων, ο αλγόριθμος λειτουργεί βάσει μόνο των εκδηλώσεων που έχει επισκεφθεί. Το API επιστρέφει επομένως πεδίο `strategy` με δύο δυνατές τιμές, και η διεπαφή προσαρμόζει ανάλογα τον τίτλο και την επεξήγηση, ώστε ο χρήστης να κατανοεί σε τι βασίζεται η πρόταση:

strategy	Τίτλος	Επεξήγηση στη διεπαφή
matrix_factorization	Προτεινόμενα για εσένα	«Με βάση τις κρατήσεις σου και χρήστες με παρόμοια ενδιαφέροντα.»
cold_start_visits	Μπορεί να σου αρέσουν	«Με βάση τις εκδηλώσεις που έχεις δει.»

Από τις υποψήφιες εκδηλώσεις αποκλείονται όσες ο χρήστης έχει ήδη δει ή κρατήσει, όσες διοργανώνει ο ίδιος, όσες έχουν ήδη πραγματοποιηθεί και όσες δεν είναι δημοσιευμένες. Συνέπεια αυτού είναι ότι το **κενό σύνολο προτάσεων αποτελεί απολύτως φυσιολογική απάντηση**· η διεπαφή τότε δεν εμφανίζει καθόλου το σχετικό τμήμα, αντί να δείξει άδειο πλαίσιο.

7.5 Αξιολόγηση στο dataset του e-class

Η §13 αναφέρει ότι «για το σκοπό αυτό παρέχεται dataset στο e-class». Σε μια νέα εγκατάσταση τα δεδομένα της ίδιας της εφαρμογής είναι ελάχιστα, οπότε ο αλγόριθμος αξιολογήθηκε σε αυτό το dataset με το σενάριο `services/evaluate_recommender.py`, το οποίο εκπαιδεύει **τον ίδιο κώδικα** που χρησιμοποιεί η εφαρμογή.

Χρησιμοποιήθηκε το `event_interest.csv`: 13.983 ζεύγη χρήστη-εκδήλωσης, 1.970 χρήστες και 8.119 εκδηλώσεις, με πυκνότητα πίνακα μόλις 0,087%. Είναι το μόνο αρχείο όπου και τα δύο αναγνωριστικά είναι συμπληρωμένα· στο `event_attendees.csv` λιγότερο από 1% των γραμμών έχει χρήστη, ενώ το `events.csv` δεν περιέχει τίτλους ή χώρους. Κάθε γραμμή αντιστοιχίστηκε με τα βάρη της εφαρμογής: η προβολή μετράει όπως μια επίσκεψη (1), το «ενδιαφέρομαι» όπως μια κράτηση (+5), και το ρητό «δεν ενδιαφέρομαι» — που η εφαρμογή δεν διαθέτει — ως 0.

Τα δεδομένα χωρίστηκαν σε σύνολα εκπαίδευσης και ελέγχου (80/20). Οι υπερπαράμετροι επιλέχθηκαν σε **ξεχωριστό σύνολο επικύρωσης**, ώστε το σύνολο ελέγχου να μην επηρεάζει την επιλογή. Κάθε τελική μέτρηση επαναλήφθηκε σε **πέντε διαφορετικούς τυχαίους διαχωρισμούς**: μια διαφορά μεταξύ μοντέλων έχει νόημα μόνο αν ξεπερνά τη διακύμανση από διαχωρισμό σε διαχωρισμό. Μετρήθηκαν δύο διαφορετικά ερωτήματα:

1. **Από τις εκδηλώσεις που προβλήθηκαν στον χρήστη, ποιες τον ενδιέφεραν;** Μέτρο η AUC πάνω στα ζεύγη ελέγχου του dataset.
2. **Από εκδηλώσεις που δεν έχει δει, ποιες να του προταθούν;** Αυτό κάνει η εφαρμογή. Για κάθε εκδήλωση ελέγχου που ενδιέφερε τον χρήστη, μετράται πόσο ψηλά την κατατάσσει το μοντέλο ανάμεσα σε 99 τυχαίες εκδηλώσεις που δεν έχει δει: HR@10 (ποσοστό που μπαίνει στις 10 πρώτες) και AUC.

Μοντέλο	AUC προβληθέντων	HR@10 αθέατων	AUC αθέατων
Τυχαία κατάταξη / γενικός μέσος όρος	0.500	0.092 ± 0.004	0.500
Αρχικές προεπιλογές (k=8, λ=0.05, 60 εποχές, χωρίς αρνητικά)	0.590 ± 0.012	0.207 ± 0.006	0.682 ± 0.003
Μόνο biases, χωρίς αρνητικά, 5 εποχές	0.612 ± 0.010	0.295 ± 0.015	0.692 ± 0.004
Μόνο biases, με αρνητικά, 20 εποχές	0.595 ± 0.010	0.362 ± 0.018	0.761 ± 0.005
Προεπιλογές εφαρμογής (k=8, λ=0.1, 20 εποχές, 1 αρνητικό)	0.598 ± 0.012	0.364 ± 0.016	0.762 ± 0.006

Πίνακας 7.1 — Μέσος όρος ± τυπική απόκλιση σε πέντε διαχωρισμούς. Πλήρη αποτελέσματα, μαζί με τη ρύθμιση στο σύνολο επικύρωσης: [docs/recommender_evaluation.md](#).

Από τα αποτελέσματα προκύπτουν τρία συμπεράσματα.

Οι αρχικές προεπιλογές έκαναν overfitting. Με 60 εποχές το σφάλμα στα δεδομένα εκπαίδευσης έπεφτε στο 0,49, ενώ σε άγνωστα ζεύγη το RMSE (2,309) ήταν χειρότερο ακόμη και από τον σκέτο γενικό μέσο όρο (2,242). Το μοντέλο αποστήθιζε αντί να γενικεύει. Η μείωση σε 20 εποχές το διόρθωσε.

Τα αρνητικά δείγματα είναι η σημαντικότερη αλλαγή. Στο ερώτημα της εφαρμογής, το HR@10 σχεδόν διπλασιάστηκε (από 0,207 σε 0,364, με το τυχαίο στο 0,09) και η AUC ανέβηκε από 0,682 σε 0,762. Το τίμημα είναι μια μικρή πτώση στο πρώτο ερώτημα (0,598 έναντι 0,612 του καλύτερου μοντέλου), γιατί εκεί εξετάζονται μόνο εκδηλώσεις που ήδη προβλήθηκαν στον χρήστη, ενώ τα αρνητικά δείγματα μαθαίνουν τη διάκριση «είδε – δεν είδε». Επειδή η εφαρμογή προτείνει μόνο εκδηλώσεις που ο χρήστης δεν έχει δει, προτεραιότητα δόθηκε στο δεύτερο ερώτημα. Η RMSE δεν περιλαμβάνεται στη σύγκριση: με αρνητικά δείγματα αλλάζει η κατανομή των στόχων, οπότε οι τιμές της δεν είναι συγκρίσιμες.

Σε αυτό το dataset, τα λανθάνοντα χαρακτηριστικά δεν πρόσθεσαν μετρήσιμη βελτίωση πάνω από τις προκαταλήψεις (0,364 έναντι 0,362, με τυπική απόκλιση 0,016). Η εξήγηση είναι η αραιότητα: περίπου 1,7 αλληλεπιδράσεις ανά εκδήλωση, και μόνο το 49% των ζευγών ελέγχου αφορά χρήστη και εκδήλωση που εμφανίστηκαν στην εκπαίδευση — για τα υπόλοιπα, το μοντέλο αναγκαστικά πέφτει στο $\mu + b_u$. Τα χαρακτηριστικά παραμένουν στο μοντέλο γιατί, όταν τα δεδομένα έχουν δομή, κάνουν τη διαφορά: στο ελεγχόμενο πείραμα της §7.3 οι εκδηλώσεις ήταν εξίσου δημοφιλείς, άρα οι προκαταλήψεις δεν μπορούσαν να τις ξεχωρίσουν, και μόνο τα χαρακτηριστικά έδωσαν σωστή κατάταξη σε 10 από 10 εκτελέσεις.

8. Εξαγωγή δεδομένων σε XML και JSON

Η §12 απαιτεί ο διαχειριστής να εξάγει τις εκδηλώσεις σε XML σύμφωνα με το DTD της §7, καθώς και σε αντίστοιχη μορφή JSON. Το endpoint `GET /api/admin/events/export?format=xml|json` εξυπηρετεί και τις δύο.

Εξάγονται **όλες** οι καταστάσεις εκδηλώσεων, όχι μόνο οι δημοσιευμένες: η εκφώνηση ζητά «τις εκδηλώσεις που έχουν δημιουργηθεί από τους χρήστες», χωρίς περιορισμό δημοσίευσης. Η σειρά των στοιχείων ακολουθεί αυστηρά το DTD, το οποίο απαιτεί συγκεκριμένη διάταξη.

```
<Event EventID="EV1">
  <Title>Συναυλία Σύγχρονης Μουσικής</Title>
  <Category>Music</Category>
  <Category>Live Performance</Category>
  <EventType>Concert</EventType>
  <Venue>Θέατρο Πόλης</Venue>
  ...
  <GeoLocation Latitude="37.9838" Longitude="23.7275" />
  <StartDateTime>2026-11-14T20:30:00</StartDateTime>
  ...
  <TicketTypes>
    <TicketType TicketTypeID="T1">
      <Name>General Admission</Name>
      <Price>18.00</Price>
      <Quantity>250</Quantity>
      <Available>248</Available>
    </TicketType>
  </TicketTypes>
  <Bookings>
    <Booking BookingID="B1">
      <Attendee UserID="maria21" />
      <Time>2026-09-12T13:03:25</Time>
      <TicketTypeRef>T1</TicketTypeRef>
      <NumberOfTickets>2</NumberOfTickets>
      <TotalCost>36.00</TotalCost>
      <BookingStatus>CONFIRMED</BookingStatus>
    </Booking>
  </Bookings>
  <Organizer UserID="org_athens_events" />
  <Status>PUBLISHED</Status>
  ...
</Event>
```

Δύο σημεία χρήζουν προσοχής. Πρώτον, οι ημερομηνίες γράφονται **χωρίς κατάληξη Z**, σε αντίθεση με το υπόλοιπο API, επειδή έτσι τις ορίζει το DTD. Δεύτερον, η ελάχιστη απαίτηση `Category+` του DTD σημαίνει ότι εκδήλωση χωρίς καμία κατηγορία θα καθιστούσε ολόκληρο

το παραγόμενο έγγραφο μη έγκυρο· γι' αυτό η επικύρωση απαιτεί τουλάχιστον μία κατηγορία ήδη κατά τη δημιουργία.

9. Εγκατάσταση και εκτέλεση

9.1 Προαπαιτούμενα

Python 3.11 ή νεότερη, Node.js 18 ή νεότερη, MySQL 8, OpenSSL.

9.2 Βάση δεδομένων

```
mysql -u root -p -e "CREATE DATABASE staywebapp
                     CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci"

# Σχήμα (με τη σειρά – υπάρχουν εξαρτήσεις ξένων κλειδιών)
for t in users events event_categories ticket_types bookings \
    messages event_media event_visits routines; do
    mysql -u root -p staywebapp < db/staywebapp_${t}.sql
done

# Μεταναστεύσεις
mysql -u root -p staywebapp < db/migrations/001_event_categories_many_to_many.sql
mysql -u root -p staywebapp < db/migrations/002_messages_soft_delete.sql
```

Και οι δύο μεταναστεύσεις είναι **απαραίτητες**. Το νωτιαίο άκρο ξεκινά κανονικά και χωρίς αυτές, αφού δημιουργεί μόνο τους πίνακες που λείπουν και δεν τροποποιεί τους υπάρχοντες. Χωρίς τη 002 όμως κάθε κλήση των endpoints μηνυμάτων αποτυγχάνει, επειδή λείπουν οι στήλες διαγραφής ανά χρήστη. Το σφάλμα εμφανίζεται αμέσως μετά την είσοδο, γιατί η διεπαφή ρωτά το πλήθος των μη αναγνωσμένων μηνυμάτων.

9.3 Νωτιαίο άκρο

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt      # Windows: .venv/Scripts/pip

# Αρχείο .env στη ρίζα του έργου:
# DATABASE_URL=mysql+pymysql://root:<κωδικός>@127.0.0.1:3306/staywebapp?charset=utf8mb4
# JWT_SECRET_KEY=<τυχαία συμβολοσειρά>
# JWT_ALGORITHM=HS256
# ACCESS_TOKEN_EXPIRE_MINUTES=60
# ADMIN_USERNAME=admin
# ADMIN_PASSWORD=<κωδικός διαχειριστή>

bash make-certs.sh      # αυτο-υπογεγραμμένο πιστοποιητικό (μία φορά)
bash run.sh             # https://localhost:8000
```

Ο χρήστης-διαχειριστής δημιουργείται αυτόματα κατά την πρώτη εκκίνηση, όπως απαιτεί η §3 της εκφώνησης, με τα στοιχεία `ADMIN_USERNAME` / `ADMIN_PASSWORD` του `.env`. Αλλαγή του κωδικού στο `.env` αργότερα δεν επηρεάζει τον λογαριασμό, αφού αυτός υπάρχει ήδη στη βάση. Η διαδραστική τεκμηρίωση του API είναι διαθέσιμη στο `https://localhost:8000/docs`. Η διεύθυνση `0.0.0.0` που τυπώνει ο server σημαίνει «ακούει σε όλες τις διεπαφές» και δεν ανοίγει σε browser.

ΝΩΤΙΑΙΟ ΑΚΡΟ ΣΕ WSL, MYSQL ΣΕ WINDOWS

Στην προεπιλεγμένη λειτουργία δικτύου του WSL 2 (NAT), η διεύθυνση `127.0.0.1` μέσα στο WSL αναφέρεται στο ίδιο το WSL και όχι στα Windows, οπότε η σύνδεση στη MySQL απορρίπτεται. Η σύνδεση στη διεύθυνση των Windows αποτυγχάνει επίσης, γιατί ο χρήστης της MySQL επιτρέπεται μόνο από `localhost` (σφάλμα 1130). Η λύση είναι η γραμμή `networkingMode=mirrored` στο `%UserProfile%\wslconfig` και επανεκκίνηση με `wsl -- shutdown`: το WSL μοιράζεται τότε το δίκτυο των Windows και δεν χρειάζεται καμία αλλαγή στο `.env`.

9.4 Μετωπιαίο άκρο

```
npm install
npm run dev          # https://localhost:5173
```

Επειδή το πιστοποιητικό είναι αυτο-υπογεγραμμένο, ο browser ζητά ρητή αποδοχή της εξαίρεσης την πρώτη φορά. Οι κλήσεις προς `/api` προωθούνται εσωτερικά στο νωτιαίο άκρο, οπότε αρκεί **μία** αποδοχή.

9.5 Δεδομένα επίδειξης

Μια νέα εγκατάσταση ξεκινά με άδεια βάση. Για να φαίνεται κάθε λειτουργία, το σενάριο `db/seed_demo.py` δημιουργεί 8 χρήστες, 12 εκδηλώσεις σε πέντε πόλεις με φωτογραφίες, 15 κρατήσεις, επισκέψεις και μηνύματα. Κάθε εγγραφή υπάρχει για να αναδεικνύει μια συγκεκριμένη απαίτηση της εκφώνησης:

```
.venv/bin/python -m db.seed_demo      # γεμίζει τη βάση
.venv/bin/python -m db.seed_demo --reset  # σβήνει και ξαναφτιάχνει τα δεδομένα επίδειξης
```

Χρήστης	Κατάσταση	Τι αναδεικνύει
nikos_org	εγκεκριμένος	Διοργανωτής με πρόχειρη, ακυρωμένη (§7γ), δημοσιευμένη χωρίς κρατήσεις που διαγράφεται, και εκδήλωση που έχει ήδη ξεκινήσει (§9)
eleni_org	εγκεκριμένη	Διοργανώτρια· πρόχειρη εκδήλωση ορατή μόνο σε αυτήν
maria	εγκεκριμένη	Κρατήσεις σε μουσικές εκδηλώσεις → εξατομικευμένες συστάσεις (§13), μη αναγνωσμένα μηνύματα (§10)
kostas	εγκεκριμένος	Κοινές κρατήσεις με τη maria, από τις οποίες μαθαίνει ο αλγόριθμος
giannis	εγκεκριμένος	Μήνυμα ακύρωσης στα εισερχόμενα (§10)
sofia	εγκεκριμένη	Μόνο επισκέψεις, καμία κράτηση → ψυχρή εκκίνηση (§13)
alexis	σε αναμονή	Αίτηση εγγραφής προς έγκριση από τον διαχειριστή (§4)
dimitra	απορρίφθηκε	Αδυναμία εισόδου μετά από απόρριψη (§2)

Κοινός κωδικός: `demo1234`. Τον διαχειριστή και όσους χρήστες δεν ανήκουν στα δεδομένα επίδειξης το σενάριο δεν τους αγγίζει. Οι ημερομηνίες υπολογίζονται σχετικά με τη στιγμή εκτέλεσης, ώστε οι εκδηλώσεις να είναι μελλοντικές· πριν από μια παρουσίαση το σενάριο εκτελείται ξανά με `--reset`. Η επαναφορά σβήνει τις εκδηλώσεις επίδειξης μαζί με όλες τις κρατήσεις τους, και όσες έκαναν άλλοι χρήστες από τη διεπαφή.

Το dataset του e-class δεν χρησιμοποιήθηκε για αυτόν τον σκοπό: δεν περιέχει τίτλους, χώρους ή τύπους εισιτηρίων, οι ημερομηνίες του είναι γύρω στο 2012 και οι χρήστες του δεν έχουν ονόματα, email ή ΑΦΜ. Αξιοποιήθηκε εκεί όπου προορίζεται, στην αξιολόγηση του αλγορίθμου (§7.5).

9.6 Αξιολόγηση του αλγορίθμου

Το dataset (1,5 GB) δεν περιλαμβάνεται στο παραδοτέο λόγω μεγέθους. Τοποθετείται ως `dataset/rel_event_csvs/` και η αξιολόγηση της §7.5 αναπαράγεται με:

```
.venv/bin/python -m services.evaluate_recommender
```

Οι σπόροι τυχαιότητας είναι σταθεροί, οπότε τα αποτελέσματα είναι αναπαραγώγιμα. Η έξοδος της τελευταίας εκτέλεσης βρίσκεται στο `docs/recommender_evaluation.md`.

10. Σχεδιαστικές αποφάσεις και παραδοχές

10.1 Το συμβόλαιο υπερισχύει

Καθ' όλη την ανάπτυξη ίσχυσε ένας κανόνας: όπου μια βιαστική υλοποίηση οδηγούσε σε κάτι διαφορετικό από το συμβόλαιο, υλοποιούσαμε το συμβόλαιο και το σημειώναμε. Στις τρεις περιπτώσεις όπου το API κατέληξε *υπερσύνολο* της προδιαγραφής — προσθέτοντας πεδία ή endpoints χωρίς να αλλάζει υπάρχοντα — και στη μία όπου η βάση δεν επέτρεπε αυτό που όριζε το συμβόλαιο, η απόκλιση καταγράφηκε στο changelog (έκδοση v1.1). Η έκδοση v1.2 προέκυψε από την αντίστροφη κατεύθυνση: έλεγχος συμμόρφωσης με την ίδια την εκφώνηση εντόπισε σημεία όπου το συμβόλαιο ήταν αυστηρότερο ή ελλιπέστερο από αυτήν — ο κανόνας διαγραφής (§4.4), οι φωτογραφίες (§10.4), η υποχρεωτική γεωγραφική θέση (§10.7) και οι κρατήσεις μετά την έναρξη — και τα ευθυγράμμισε.

10.2 Το `eventId` είναι υποχρεωτικό σε κάθε μήνυμα

Η στήλη `messages.fk_event_id` ορίστηκε `NOT NULL`. Κάθε μήνυμα ανήκει σε μία εκδήλωση — παραδοχή που συμφωνεί με την §10, όπου η συνομιλία γεννιέται από κράτηση. Δεν υπάρχει «γενικό» μήνυμα εκτός πλαισίου εκδήλωσης.

10.3 Οι τύποι εισιτηρίων ταυτοποιούνται με το όνομα

Το σώμα του αιτήματος ενημέρωσης είναι πανομοιότυπο με αυτό της δημιουργίας και **δεν περιέχει αναγνωριστικά** τύπων εισιτηρίων. Η αντιστοίχιση γίνεται επομένως με το όνομα: όνομα που υπάρχει ενημερώνεται, όνομα που λείπει διαγράφεται, νέο όνομα δημιουργείται. Δύο τύποι με ίδιο όνομα απορρίπτονται κατά την επικύρωση, αφού θα ήταν αδύνατο να αντιστοιχιστούν.

ΣΥΝΕΠΕΙΑ

Μετονομασία τύπου εισιτηρίου ισοδυναμεί με διαγραφή και δημιουργία. Αν ο τύπος έχει κρατήσεις, η ενέργεια αποτυγχάνει με `409 SEATS_UNAVAILABLE`. Για να μη διορθώσει ο διοργανωτής ένα τυπογραφικό και φάει ανεξήγητο σφάλμα, η διεπαφή **κλειδώνει το πεδίο ονόματος** για κάθε τύπο με κρατήσεις και εξηγεί τον λόγο.

10.4 Φωτογραφίες εκδηλώσεων

Η §7α ορίζει ότι «θα αποθηκεύονται προαιρετικά μία ή περισσότερες φωτογραφίες για κάθε εκδήλωση». Το «προαιρετικά» ερμηνεύτηκε ως προαιρετικό για τον διοργανωτή και όχι για την εφαρμογή: η δυνατότητα πρέπει να υπάρχει. Ο διοργανωτής ανεβάζει έως 10 φωτογραφίες από τη φόρμα της εκδήλωσης, μέσω πολυμερούς αιτήματος (`POST /events/{id}/media`). Η πρώτη εμφανίζεται ως εξώφυλλο στην αναζήτηση και όλες σε συλλογή στη σελίδα της εκδήλωσης.

Επειδή πρόκειται για αρχεία που ορίζει ο χρήστης, ο server εφαρμόζει τρεις κανόνες:

- Ο τύπος κρίνεται από τα **πρώτα bytes** του αρχείου (JPEG, PNG, GIF, WebP) και όχι από την κατάληξη ή το `Content-Type`, που τα ορίζει ο client και ψεύδονται εύκολα. Το SVG απορρίπτεται, αφού μπορεί να περιέχει JavaScript.
- Το όνομα στον δίσκο το παράγει ο server (τυχαίο UUID). Το όνομα που έστειλε ο χρήστης δεν χρησιμοποιείται ποτέ, οπότε αποκλείονται διαδρομές όπως `../../main.py` και συγκρούσεις ονομάτων.
- Όλα ή τίποτα: αν έστω μία φωτογραφία είναι άκυρη, δεν αποθηκεύεται καμία.

Τα αρχεία γράφονται στον φάκελο `media/`, η βάση κρατά μόνο το όνομα (`event_media`) και σερβίρονται από το `/api/media/{όνομα}`. Η ενημέρωση εκδήλωσης μπορεί μόνο να αφαιρέσει φωτογραφίες: ονόματα που δεν ανήκουν ήδη στην εκδήλωση αγνοούνται, ώστε κανείς να μη «δανειστεί» αρχείο άλλης εκδήλωσης. Κατά τη διαγραφή εκδήλωσης τα αρχεία σβήνονται **μετά** την επιτυχή δέσμευση της συναλλαγής: αν η βάση αποτύγχανε, δεν θέλουμε εκδήλωση που υπάρχει ακόμη χωρίς τις φωτογραφίες της.

10.5 Αποθήκευση του JWT

Το token αποθηκεύεται στο `localStorage`. Η επιλογή είναι απλή και επιβιώνει σε ανανέωση σελίδας, αλλά είναι **ευάλωτη σε επιθέσεις XSS**: κακόβουλος κώδικας που θα εκτελούνταν στη σελίδα θα μπορούσε να το διαβάσει. Εναλλακτική θα ήταν cookie με σημαία `HttpOnly`, που όμως απαιτεί προστασία έναντι CSRF. Για το εύρος της εργασίας η επιλογή θεωρήθηκε αποδεκτή και καταγράφεται εδώ ρητά ως γνωστός περιορισμός.

10.6 Εκπαίδευση του μοντέλου κατά την κλήση

Το μοντέλο συστάσεων εκπαιδεύεται επιτόπου σε κάθε αίτημα. Για το μέγεθος δεδομένων μιας φοιτητικής εφαρμογής το κόστος είναι αμελητέο. Σε περιβάλλον παραγωγής η

εκπαίδευση θα εκτελούνταν περιοδικά ως εργασία παρασκηνίου και το endpoint θα διάβαζε μόνο τους αποθηκευμένους πίνακες παραγόντων.

10.7 Γεωγραφική θέση υποχρεωτική στην εγγραφή

Η §2 ορίζει ότι η σελίδα εγγραφής «θα απαιτεί» στοιχεία διεύθυνσης και γεωγραφικής τοποθεσίας. Οι συντεταγμένες είναι επομένως υποχρεωτικές και ελέγχονται ως προς τα όριά τους και στα δύο άκρα. Για να μη χρειάζεται ο χρήστης να τις αναζητήσει, η φόρμα προσφέρει κουμπί που τις συμπληρώνει από τον browser (Geolocation API, διαθέσιμο επειδή η εφαρμογή τρέχει πάνω από HTTPS).

10.8 Παρουσίαση των αποτελεσμάτων αναζήτησης

Η §8 ζητά οι εκδηλώσεις να εμφανίζονται «σε πίνακες κατάλληλα σελιδοποιημένους». Τα αποτελέσματα παρουσιάζονται σε **πλέγμα καρτών** με σελιδοποίηση, αντί για γραμμές πίνακα, ώστε να χωρούν εξώφυλλο, κατηγορίες, τιμή και διαθεσιμότητα και να παραμένουν ευανάγνωστα σε κινητό. Η σελιδοποίηση γίνεται στον server (`page` / `pageSize`), όπως ορίζει το σύμβολαιο.

10.9 Πληρωμές εκτός εμβέλειας

Η εκφώνηση περιγράφει ηλεκτρονικές κρατήσεις εισιτηρίων και θέσεων, με έλεγχο διαθεσιμότητας και «συνολικό κόστος κράτησης», χωρίς να αναφέρει πληρωμή. Η εφαρμογή επομένως **δεν υλοποιεί διαδικασία πληρωμής**. Το κόστος υπολογίζεται αποκλειστικά στον server (τιμή × πλήθος εισιτηρίων, σε δεκαδική αναπαράσταση), δεν γίνεται δεκτό από τον client, και η κράτηση καταχωρείται απευθείας ως `CONFIRMED` μετά την επιβεβαίωση του χρήστη.

Το σχήμα είναι έτοιμο για μελλοντική επέκταση: η κατάσταση `PENDING` που προβλέπει το DTD για τις κρατήσεις θα μπορούσε να περιγράφει μια κράτηση που περιμένει πληρωμή, με τη μετάβαση σε `CONFIRMED` να γίνεται μετά την επιβεβαίωση από πάροχο πληρωμών.

11. Επίλογος

11.1 Σύνοψη

Η εφαρμογή υλοποιεί το σύνολο των λειτουργικών απαιτήσεων της εκφώνησης: εγγραφή με έγκριση διαχειριστή, δημιουργία και διαχείριση εκδηλώσεων με έλεγχο χωρητικότητας και φωτογραφίες, αναζήτηση με πολλαπλά κριτήρια και σελιδοποίηση, κρατήσεις με προστασία από υπερκράτηση, επικοινωνία διοργανωτή-συμμετέχοντα, εξαγωγή σε XML κατά το DTD και σε JSON, και σύστημα συστάσεων με Biased Matrix Factorization γραμμένο από το μηδέν.

Ο αλγόριθμος αξιολογήθηκε στο dataset του e-class με διαχωρισμό εκπαίδευσης, επικύρωσης και ελέγχου. Η αξιολόγηση οδήγησε σε δύο διορθώσεις — λιγότερες εποχές και δειγματοληψία αρνητικών — που σχεδόν διπλασίασαν την ποιότητα των προτάσεων (§7.5). Η εγκατάσταση συνοδεύεται από σενάριο δεδομένων επίδειξης, ώστε κάθε απαίτηση να μπορεί να παρουσιαστεί σε λειτουργία (§9.5).

11.2 Δυσκολίες και πώς αντιμετωπίστηκαν

Η ένωση δύο άκρων που δεν είχαν ποτέ συναντηθεί

Η μεγαλύτερη πηγή ρίσκου ήταν ότι τα δύο άκρα αναπτύχθηκαν εντελώς παράλληλα και δοκιμάστηκαν μαζί μόνο στο τέλος. Η αντιμετώπιση ήταν το ρητό συμβόλαιο (§4.1) και, κατά την ένωση, ένα **αυτοματοποιημένο σενάριο ελέγχου** που διατρέχει ολόκληρη τη ροή: εγγραφή, έγκριση, δημιουργία, δημοσίευση, κράτηση, μήνυμα, ακύρωση, εξαγωγή, καθώς και τους ελέγχους εξουσιοδότησης. Το σενάριο εντόπισε αμέσως ό,τι δεν ταίριαζε και έγινε το δίκτυ ασφαλείας για κάθε επόμενη αλλαγή.

Ασυμβατότητες πλατφόρμας

Τα σενάρια εκκίνησης γράφτηκαν σε περιβάλλον Unix και δεν εκτελούνταν καθόλου σε Windows. Εντοπίστηκαν και διορθώθηκαν τρία ανεξάρτητα προβλήματα:

- Το εικονικό περιβάλλον της Python τοποθετεί τα εκτελέσιμα σε `Scripts/` αντί για `bin/`.
- Η κονσόλα των Windows χρησιμοποιεί κωδικοποίηση cp1252, με αποτέλεσμα η εκτύπωση ενός ελληνικού μηνύματος κατά την αρχικοποίηση να **ρίχνει τον server** — αφού είχε ήδη εγγράψει τον διαχειριστή στη βάση. Το σφάλμα έμοιαζε με πρόβλημα βάσης ενώ ήταν κωδικοποίησης. Λύθηκε ορίζοντας ρητά UTF-8 στην έξοδο.

- Το Git Bash μετέτρεπε το όρισμα `-subj "/C=GR/..."` του OpenSSL σε διαδρομή αρχείου, αφήνοντας το ιδιωτικό κλειδί χωρίς πιστοποιητικό.
- Με το νωτιαίο άκρο σε WSL και τη MySQL σε Windows, ο server δεν ξεκινούσε: το `127.0.0.1` του WSL δεν αντιστοιχεί στα Windows. Η διάγνωση έγινε σε βήματα — η υπηρεσία της MySQL έτρεχε, η θύρα ήταν ανοιχτή από τη διεύθυνση των Windows, αλλά ο χρήστης επιτρεπόταν μόνο από `localhost`. Λύθηκε με το κοινόχρηστο δίκτυο του WSL (§9.3), χωρίς να ανοιχτεί η βάση σε άλλες διευθύνσεις.

Μια μετανάστευση που δεν είχε εκτελεστεί

Η εφαρμογή ξεκινούσε κανονικά, όμως τα μηνύματα αποτύγχαναν με σφάλμα διακομιστή. Το εντόπισε το σενάριο δεδομένων επίδειξης, που ήταν το πρώτο που προσπάθησε να γράψει μήνυμα στη βάση: η μετανάστευση 002 δεν είχε εφαρμοστεί στη βάση ανάπτυξης. Επειδή ο server δημιουργεί μόνο πίνακες που λείπουν και δεν ελέγχει στήλες, η παράλειψη δεν φαινόταν στην εκκίνηση. Η εμπειρία καταγράφηκε ως ρητή προειδοποίηση στις οδηγίες εγκατάστασης (§9.2).

Ένα σφάλμα που δεν υπήρχε

Σε δοκιμές μέσω τερματικού, τα ελληνικά ονόματα επέστρεφαν από το API ως ερωτηματικά, γεγονός που υποδείκνυε λανθασμένη κωδικοποίηση στη βάση. Ο έλεγχος έδειξε ότι οι στήλες ήταν σωστά `utf8mb4` και ότι τα δεδομένα αποθηκεύονταν *ακέραια* όταν το αίτημα στελνόταν από αρχείο. Η αλλοίωση συνέβαινε στο κέλυφος, πριν καν φύγει το αίτημα. Το περιστατικό κατέγραψε ένα χρήσιμο μάθημα: **πριν διορθώσεις ένα σφάλμα, βεβαιώσου ότι το μετράς εκεί που συμβαίνει.**

Απαιτήσεις κρυμμένες σε μία πρόταση

Η δυνατότητα διαγραφής μηνυμάτων αναφέρεται σε μία μόνο πρόταση της §10 και αρχικά θεωρήθηκε προαιρετική. Η προσεκτική επανανάγνωση της εκφώνησης έδειξε ότι είναι ρητή απαίτηση, και οδήγησε στη μετανάστευση 002 (§3.4). Αντίστοιχα, το σύστημα συστάσεων αντιμετωπιζόταν ως «μπόνους» έως ότου διαπιστώθηκε ο ρόλος του στη βαθμολόγηση. Και στις δύο περιπτώσεις, η πηγή αλήθειας ήταν η εκφώνηση και όχι οι δικές μας σημειώσεις.

Ένας αλγόριθμος που «δούλευε» χωρίς να μαθαίνει

Ο αλγόριθμος συστάσεων περνούσε αρχικά όλους τους ελέγχους: το σφάλμα στα δεδομένα εκπαίδευσης έπεφτε σταθερά και σε ένα σενάριο δοκιμής πρότεινε σωστά μουσικές εκδηλώσεις σε χρήστρια με μουσικές κρατήσεις. Η αξιολόγηση στο dataset του e-class (§7.5) αποκάλυψε δύο προβλήματα που οι έλεγχοι αυτοί δεν μπορούσαν να δουν. Πρώτον, με 60 εποχές το μοντέλο αποστήθιζε τα δεδομένα εκπαίδευσης και σε άγνωστα ζεύγη ήταν χειρότερο ακόμη και από τον σκέτο μέσο όρο. Δεύτερον, επαναλαμβάνοντας το σενάριο με

δέκα διαφορετικούς σπόρους τυχειότητας, η «σωστή» πρόταση εμφανιζόταν στις δύο από τις δέκα: η αρχική επιτυχία ήταν σύμπτωση. Η αιτία ήταν ότι ο πίνακας περιείχε μόνο θετικές παρατηρήσεις (§7.3). Τα δύο προβλήματα λύθηκαν με λιγότερες εποχές και δειγματοληψία αρνητικών. Το μάθημα: **ένας έλεγχος που περνά μία φορά δεν αποδεικνύει ότι ο αλγόριθμος μαθαίνει** — χρειάζεται μέτρηση σε δεδομένα που δεν είδε και επανάληψη με διαφορετική τυχειότητα.

11.3 Τι θα κάναμε διαφορετικά

- **Αυτοματοποιημένοι έλεγχοι από την αρχή.** Το σενάριο ελέγχου γράφτηκε κατά την ένωση· αν υπήρχε από την πρώτη εβδομάδα, οι αποκλίσεις θα είχαν εντοπιστεί όσο ήταν φθηνές.
- **Έλεγχος και στις δύο πλατφόρμες νωρίτερα.** Τα προβλήματα φορητότητας ήταν όλα μικρά, αλλά εμφανίστηκαν όλα μαζί τη στιγμή που πιέζαμε για ένωση.
- **Ταυτοποίηση τύπων εισιτηρίων με αναγνωριστικό.** Η αντιστοίχιση με το όνομα λειτουργεί, αλλά κάνει τη μετονομασία αδύνατη όταν υπάρχουν κρατήσεις. Ένα πεδίο `id` στο σώμα της ενημέρωσης θα ήταν καθαρότερο — απαιτούσε όμως αλλαγή του συμβολαίου και στις δύο πλευρές.